

Ayman-Yereem Kone
Neural Graphics Assignment 1

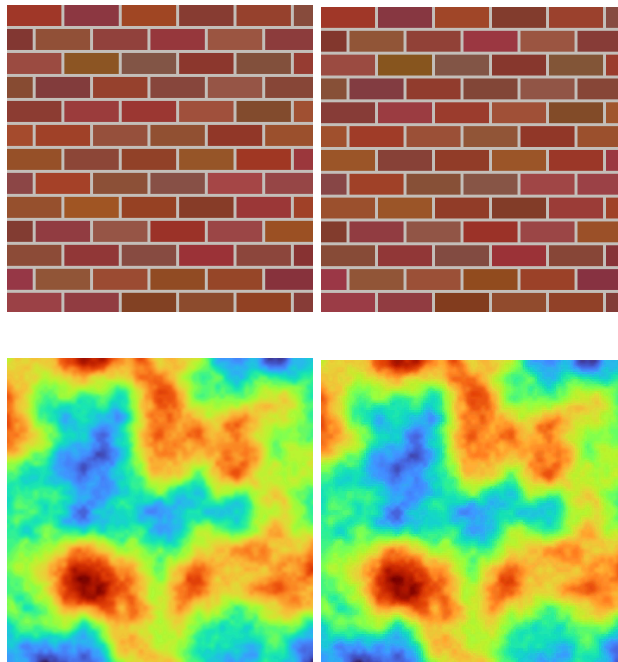
Github repo: [Yereeme/A1_NeuralGraphics_AymanKone](https://github.com/Yereeme/A1_NeuralGraphics_AymanKone)

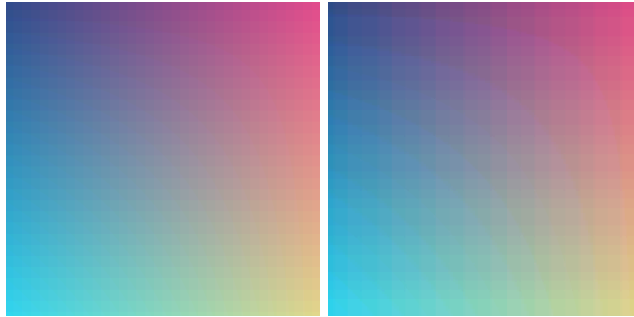
P2: S3TC Baseline

For endpoint selection, I picked c0 and c1 as the minimum and maximum color in each channel for every 4x4 block, then quantized those to RGB565. The two other palette colors (c2 and c3) just come from interpolating between c0 and c1, one third and two thirds of the way there.

Results:

Texture	PSNR (dB)	Compression Ratio
gradient.png	43.03	6.0
bricks.png	41.39	6.0
clouds.png	35.95	6.0





Original png (left) vs Reconstructed png(Right)

Where S3TC breaks down: quality gets worse as the texture has more high frequency detail in it. Gradient is smooth and low frequency so it reconstructs almost perfectly, but clouds has fine detail changing at a lot of different scales, so it loses the most quality. Makes sense since each 4x4 block only has 4 fixed colors to work with, and that's not enough to capture that much variation packed into a small area.

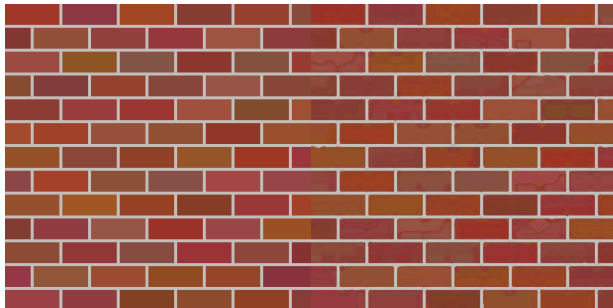
Why RGB565 gives green an extra bit: our eyes are just more sensitive to changes in green than in red or blue, so giving green 6 bits instead of 5 cuts down visible banding more per bit spent than it would for the other two channels.

P6: Results & Comparison

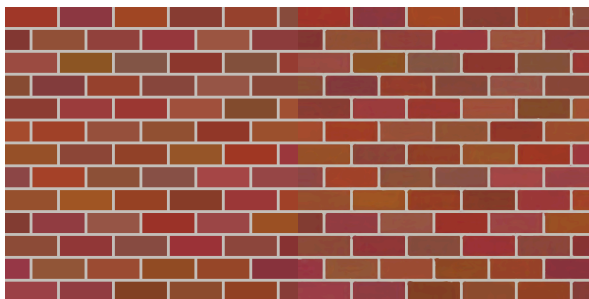
I trained three architectures (Small: one grid at resolution 64, feat_dim 2. Medium: grids at 16, 32, 64, feat_dim 2. Large: grids at 16, 32, 64, 128, feat_dim 4. All three use the same 2x64 MLP) on each of the three given textures, so 9 runs total, 2000 steps each.

Results:

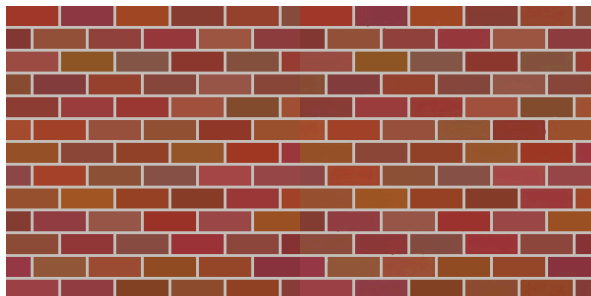
Brick.png small



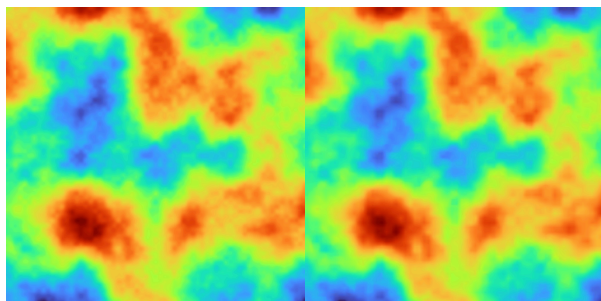
Brick.png medium



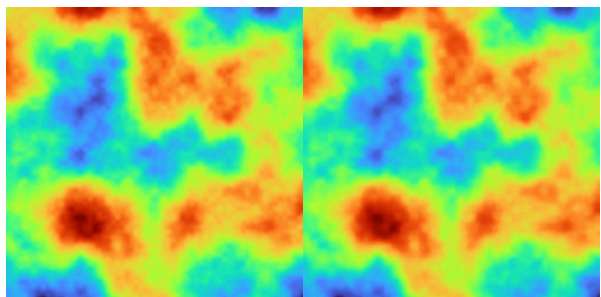
Brick.png Large



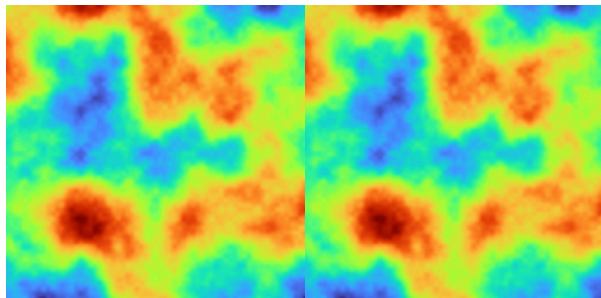
small

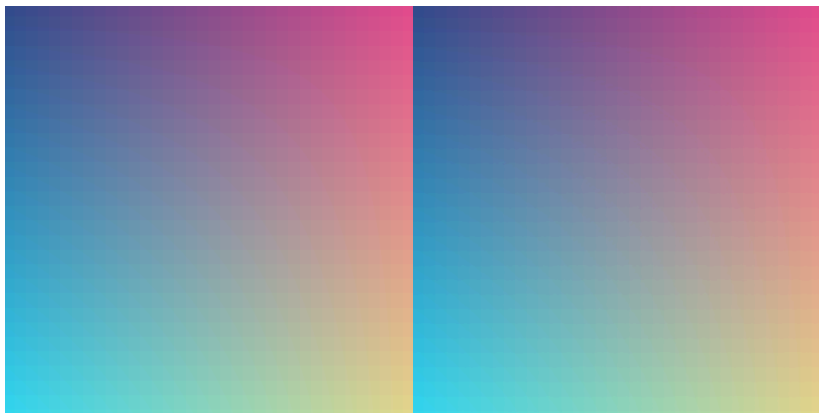
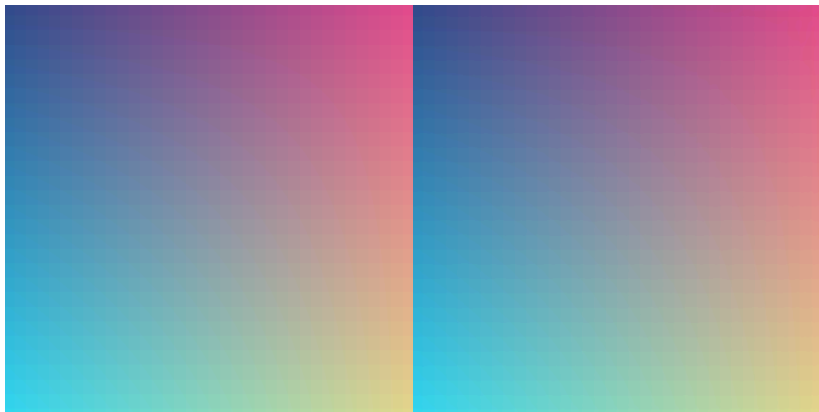
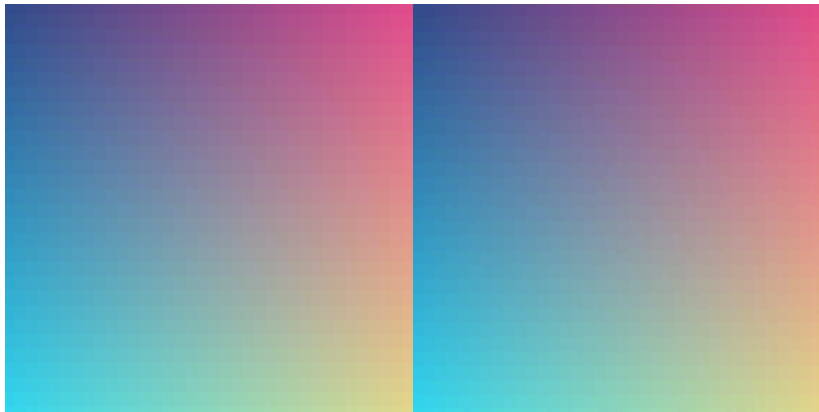


medium



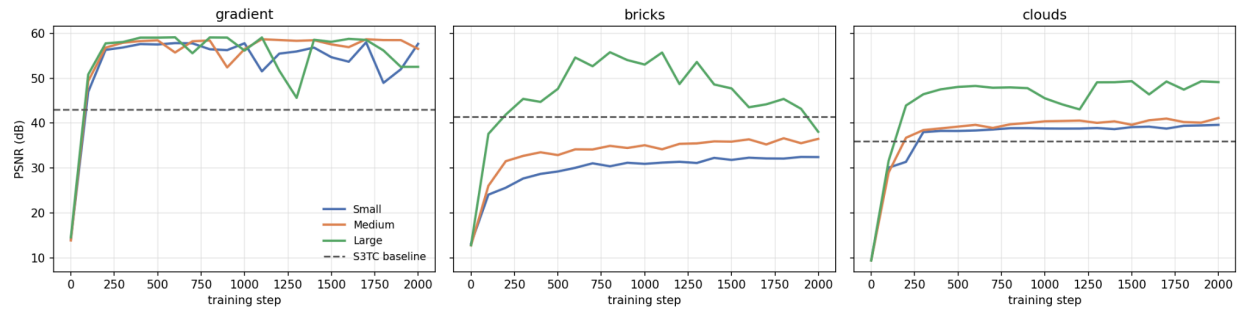
large



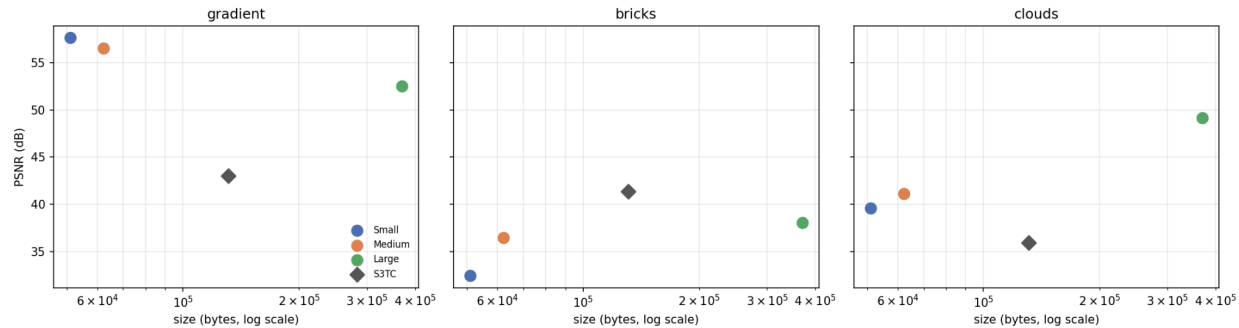


Texture	Architecture	Final PSNR (dB)	Size (bytes)	Compression Ratio
gradient.png	Small	57.64	50,956	15.4x
gradient.png	Medium	56.53	62,220	12.6x
gradient.png	Large	52.53	369,932	2.1x
bricks.png	Small	32.44	50,956	15.4x
bricks.png	Medium	36.49	62,220	12.6x
bricks.png	Large	38.07	369,932	2.1x
clouds.png	Small	39.61	50,956	15.4x
clouds.png	Medium	41.13	62,220	12.6x
clouds.png Small	Large	49.13	369,932	2.1x

P6: PSNR over training, 3 architectures x 3 textures (vs. S3TC baseline)



P6: Size vs. quality, 3 architectures x 3 textures, vs. S3TC



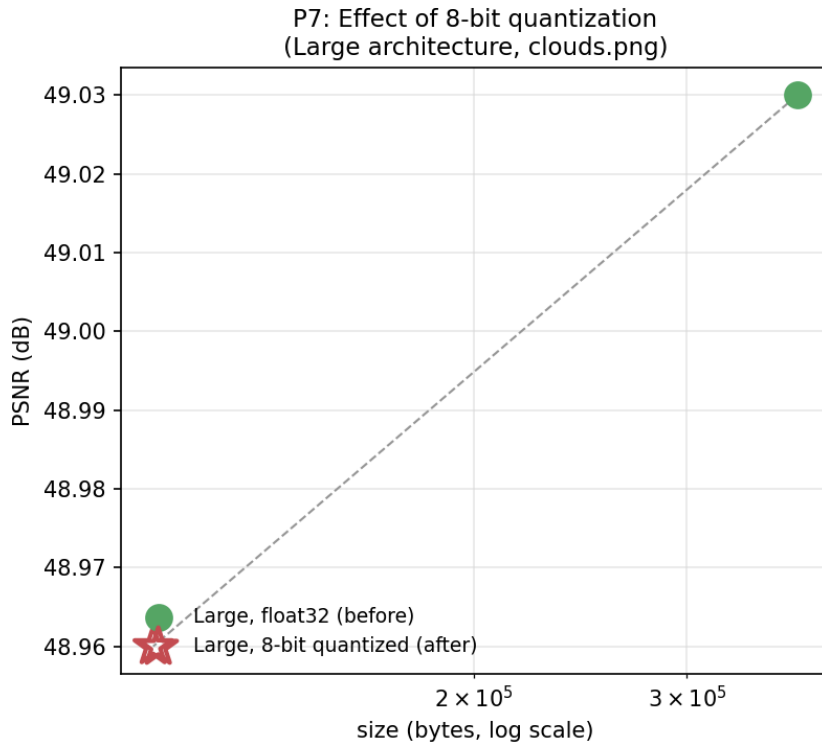
Compared to the S3TC baseline from P2 (gradient 43.03 dB, bricks 41.39 dB, clouds 35.95 dB, all at 6.0x), the neural method beats S3TC on PSNR for every texture, and usually compresses harder too. On clouds.png for example, even the Small model gets 39.61 dB at 15.4x compression, which already beats S3TC's 35.95 dB at only 6.0x.

Best architecture per texture: for gradient.png, Small wins. It's a smooth, low frequency image so one grid is already enough to represent it, and adding more grids (Medium, Large) just adds size and makes training harder without actually helping, which is probably why Large does worst here. For bricks.png and clouds.png, Large wins instead, since those textures have real high frequency detail (mortar lines, cloud structure) that a single grid can't hold. The gap between Small and Large gets bigger for both (bricks goes from 32.44 to 38.07 dB, clouds goes from 39.61 to 49.13 dB). Basically, extra capacity only helps if the texture actually has enough detail that needs it.

P7: Quantization

I took the Large architecture trained on clouds.png and quantized its grid features from float32 down to 8 bits. For each stored array I take its min and max, compute $\text{scale} = (\text{hi} - \text{lo}) / 255$, then round every value to the nearest of 256 buckets with $q = \text{round}((x - \text{lo}) / \text{scale})$, and decode back with $\hat{x} = \text{lo} + q * \text{scale}$. I left the MLP as float32 since the assignment said quantizing it is optional and it's tiny anyway.

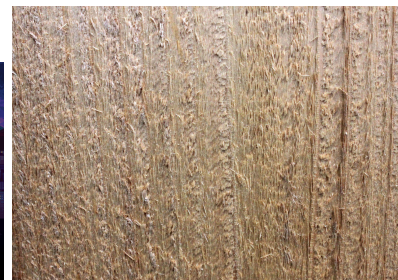
Results:



PSNR before quantization was 49.03 dB, and after it was 48.96 dB, so basically no visible drop (0.07 dB). But the size went from 369,932 bytes down to 108,812 bytes, so the compression ratio went from 2.13x to 7.23x. Makes sense because the grid values were sitting in a pretty narrow range to begin with, so rounding them into 256 buckets barely loses anything the network actually needed. Float32 was just way more precision than it needed in the first place, and that's exactly the slack quantization is taking advantage of.

P8: Your Own Textures

I picked three textures that behave pretty differently from each other: a close up wood grain photo (organic, has a direction to it, medium to high detail), a screenshot from the game Katana Zero (pixel art, sharp edges, big flat color areas), and a close up grass photo (organic, dense, high frequency, no repeating pattern).



Results:

Texture	Architecture	Final PSNR (dB)	Size (bytes)	Compression Ratio
wood	Small	19.63	51 KB	384.6x
wood	Medium	19.97	62 KB	314.9x
wood	Large	22.63	370 KB	53.0x
Katana Zero	Small	24.54	51 KB	122.1x
Katana Zero	Medium	25.07	62 KB	100.0x
Katana Zero	Large	28.47	370 KB	16.8x
grass	Small	16.12	51 KB	144.7x
grass	Medium	16.51	62 KB	118.5x
grass	Large	18.63	370 KB	19.9x

All three of these get a lot lower PSNR than the provided textures (16 to 28 dB here versus 32 to 58 dB before). That's because these are full size real photos and screenshots with way more pixels in them, so there's more actual detail packed in for the same size model to represent. The model's size doesn't change based on the image's resolution, so a bigger, more detailed image is just harder to fit into the same amount of space. That's also why the compression ratios are so high here (up to 384x for wood on Small), since raw image size scales with resolution but the neural size doesn't.

Large wins on all three, and the gap is biggest on Katana Zero (24.54 to 28.47 dB going from Small to Large), which lines up with what I saw in P6, since sharp edges and fine detail (pixel art, wood grain, grass blades) benefit the most from having multiple grid resolutions. Wood has the smallest gap between architectures, probably because its grain is more repetitive and lower frequency than the other two, so even Small can represent it decently with just one grid.